SAP Data Governance — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — SAP Data Governance: Microsoft Purview 5

What you get . . . 5

What this delivers . . . 5

The deployment model . . . 5

Outcome . . . 6

1. Solution Overview — SAP Data Governance: Microsoft Purview 6

What this solution is . . . 6

Why this matters . . . 6

The SAP sources this covers . . . 6

The Data Map, collections, and roles . . . 7

Credentials, drivers, and network — safe by design . . . 7

Licensing and the evolving portal . . . 8

What this is not . . . 8

The deployment model . . . 8

2. Requirements Guide — SAP Data Governance: Microsoft Purview 8

2.1 A Microsoft Purview account (Data Map) provisioned on an Azure subscription . . . 8

2.2 SAP connector availability confirmed for your tenant (ECC / S/4HANA / BW / HANA) . . . 9

2.3 Least-privilege data-governance roles on the Data Map collection . . . 9

2.4 An SAP system inventory in scope, with owners . . . 9

2.5 A self-hosted integration runtime (SHIR) with line of sight to SAP . . . 9

2.6 The SAP driver and Java runtime on the SHIR machine (your SAP entitlement) . . . 10

2.7 A least-privilege SAP read login, secret in Azure Key Vault — never in code . . . 10

2.8 A classification approach (system + any custom classifications) . . . 10

2.9 A pilot scope and change control . . . 11

Checklist . . . 11

Global Requirements — Microsoft Purview Solutions 11

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 12

2. Purview roles (least-privilege) and portal access . . . 12

3. Security & Compliance PowerShell + Graph toolchain . . . 12

4. Data sources & scope in scope . . . 13

5. Compliance, records, privacy & legal stakeholders . . . 13

6. A pilot scope and change control . . . 13

3. Implementation Guide — SAP Data Governance: Microsoft Purview 14

Step 1 — Assess current state (read-only) . . . 14

Step 2 — Design the collection and role model . . . 14

Step 3 — Stand up the scanner: SHIR + SAP driver + Java (safe by design) . . . 14
Step 4 — Establish the SAP credential path (Key Vault, safe by design) . . . 15
Step 5 — Record the pilot scope . . . 15
Step 6 — Register the pilot SAP source into the Data Map . . . 15
Step 7 — Configure and pilot the scan (bounded scope, pilot-first) . . . 15
Step 8 — Review scanned assets and classifications . . . 16
Step 9 — Assign ownership on the SAP assets . . . 16
Step 10 — Validate against the pilot gate . . . 16
Step 11 — Expand across the SAP estate (enforce) . . . 16
5. Testing Guide — SAP Data Governance: Microsoft Purview 17
Test 1 — Account and roles are ready . . . 17
Test 2 — The SAP connector is available . . . 17
Test 3 — The SHIR and SAP driver are in place . . . 17
Test 4 — The SAP credential path works (Key Vault + Test connection) . . . 17
Test 5 — The scan succeeds and returns assets . . . 17
Test 6 — SAP assets are classified . . . 17
Test 7 — Scanned SAP assets have owners . . . 18
Test 8 — Consolidated readiness gate . . . 18
Pass/fail gate . . . 18
6. Troubleshooting Guide — SAP Data Governance: Microsoft Purview 18
The SAP connector isn't offered in Register . . . 18
`DGSAP-Discovery.ps1` reports the endpoint as GAP (not reachable) . . . 19
No self-hosted integration runtime (SHIR) shows up . . . 19
Test connection fails: SAP driver / Java not found . . . 19
Test connection fails: authentication or authorization (SAP side) . . . 19
Test connection fails: network path to SAP . . . 19
Scan runs but returns no assets (or far fewer than expected) . . . 19
Scan Succeeded but assets are unclassified . . . 20
The scan loads the SAP system or collides with a batch window . . . 20
`-WhatIf` printed a plan but nothing was registered . . . 20
Roles look wrong or a curator can't classify . . . 20
7. Best Practices — SAP Data Governance: Microsoft Purview 20
8. Deliverables Summary — SAP Data Governance: Microsoft Purview 22
Documentation . . . 22
Named artifacts . . . 22
PowerShell scripts . . . 22

What the buyer receives 23
Outcome 23
Appendix A — PowerShell Scripts 24

Executive Summary — SAP Data Governance: Microsoft Purview

Most data-governance efforts stop at Microsoft 365, and SAP — customer master data, financial documents, HR and payroll records, material and vendor data — sits outside the map, unclassified and ungoverned. You cannot protect, retain, or grant access to data you cannot see. This solution closes that blind spot: it **registers, scans, and classifies your SAP sources into the Microsoft Purview Data Map**, so SAP becomes a mapped, classified, owned part of the same governed estate as the rest of your data.

This is a complete, do-it-today package written for an engineer bringing SAP into Purview for the first time. It delivers the SAP connector workflow end-to-end — the **self-hosted integration runtime**, the **SAP-provided driver** (SAP JCo for ECC/S/4HANA/BW, HANA JDBC for HANA), a **least-privilege SAP read user** whose secret lives in **Azure Key Vault**, a bounded pilot scan, classification review, and ownership — with every artifact named once and reused everywhere, and every scan proven on a bounded pilot before the SAP estate is touched. Because the Purview connectors are evolving and consumption-billed, the guide cites the method and flags every capability "verify your licensing/tenant" rather than asserting a connector, a driver version, or a price as permanent.

What you get

• A plain-language Solution Overview of the SAP sources — SAP ECC / S/4HANA / BW (over RFC using the

SAP JCo driver on a SHIR) and SAP HANA (over JDBC) — how they scan and classify SAP **metadata**, and how they complement Microsoft 365 Purview.

• A Requirements Guide with verify / implement-if-missing for every prerequisite: the Purview account,

connector availability, least-privilege collection roles, the SAP system inventory with owners, the SHIR, the SAP driver + Java runtime, the least-privilege SAP login held in Key Vault, a classification approach, and the pilot scope.

• An Implementation Guide of precise, ordered steps: design the collection/role model, stand up the

SHIR + SAP driver, establish the SAP credential safely, register the SAP source, **pilot-scan** a bounded set, review classifications, assign ownership, validate against the gate, then expand with scheduled scans.

• Five PowerShell tools for readiness/role discovery, pilot-scope capture, SAP source registration and

scan configuration (`-WhatIf` by default), a scan-and-classification coverage report, and a consolidated readiness-gate check — authentic Purview data-plane REST automation with **no credentials in code** (the SAP secret is referenced by name only).

• Testing and Troubleshooting guides with explicit pass/fail gates, plus field-proven Best Practices.

What this delivers

SAP data governed from Purview: SAP systems registered into the Data Map, scanned and classified, and owned — validated on a pilot before you scale.

The deployment model

Assess → Design → Configure → **Pilot (bounded scan)** → Validate → Expand — with a pilot gate before any SAP-wide scanning.

Outcome

The SAP blind spot closed — SAP systems discoverable, classified, owned, and governed in the same Purview Data Map as the rest of the estate — with regulatory, records, and access determinations left to the owners who make them.

1. Solution Overview — SAP Data Governance: Microsoft Purview

What this solution is

Microsoft Purview has two related but distinct sides. The **compliance** side (labels, DLP, retention, insider risk) governs Microsoft 365 content. This solution implements a focused slice of the **data governance** side: **registering, scanning, and classifying your SAP data into the Purview Data Map**, so SAP stops being a governance blind spot and becomes a mapped, classified, cataloged part of your estate. It does not re-implement the whole Data Map / Unified Catalog build; it delivers the **SAP-source connector workflow** end-to-end: register the source, stand up the scanner and driver path, run a bounded pilot scan, review classifications, then expand.

> ***Boundary.*** *This solution provides the technical method to *register, scan, classify, and catalog* SAP sources. It does **not** make the business or regulatory determinations — which SAP data is "confidential", what the retention or records schedule is, or who may access it — those belong to the organization's data owners, stewards, compliance, privacy, and legal functions. The solution gives them governed, discoverable SAP metadata to make those decisions against.*

Why this matters

Most organizations govern Microsoft 365 well and their SAP estate not at all. Customer master data, financial documents, HR/payroll records, and material/vendor data all live in SAP — and none of it appears in the data map. Nobody can answer "where does PII live in SAP?", "which SAP tables hold regulated data?", or "who owns this data source?" — because SAP was never registered, scanned, or classified. Ungoverned data is un-securable data: you cannot protect, retain, or grant access to something you cannot see. This solution brings SAP into the same Purview Data Map that already governs the rest of the estate — cataloging the SAP **metadata** (systems, tables/objects, fields) and applying classifications so SAP becomes discoverable and sensitivity-aware.

The SAP sources this covers

This solution stands up the SAP connector workflow. The specific SAP source types offered, their supported versions, and their exact configuration are **evolving** in Microsoft Purview — every capability below is flagged "verify your licensing/tenant". Microsoft has historically documented SAP source connectors including **SAP ECC**, **SAP S/4HANA**, **SAP BW**, and **SAP HANA**. **Confirm the current list, supported versions, and driver requirements for your tenant** in Microsoft's documentation and the

Purview registration UI before you plan.

1. **SAP ECC / SAP S/4HANA / SAP BW** — Purview reaches these application systems over **RFC** using

the **SAP Connector (SAP JCo — the Java Connector,** `sapjco3.jar` **+** `sapjco3.dll`**)** installed on the scanning machine, plus a **64-bit Java runtime**. The scan authenticates with an **SAP user** (username/password) whose secret is stored in **Azure Key Vault** and referenced **by name**, then reads **metadata** (the data dictionary — tables, structures, fields) and applies **classifications** (system classifications plus any custom classification you define).

1. **SAP HANA** — Purview connects to the HANA database with the **SAP HANA JDBC driver (**`ngdbc.jar`**)**

on the scanning machine, again authenticating with a Key-Vault-referenced database login and enumerating schemas/tables/columns for classification.

> ***Metadata, not bulk data extraction.*** *These SAP scans read the* ***catalog/metadata*** *(system, tables, fields) and sample to classify — they map and classify SAP; they are not a data-export tool.*

The Data Map, collections, and roles

SAP sources register into the **Data Map** under a **collection** hierarchy that also carries access control. Each collection carries its own role assignments: **Collection admins** (manage the collection and its roles), **Data source admins** (register sources and run scans), **Data curators** (edit catalog metadata and classifications), and **Data readers** (browse the catalog). Least-privilege role assignment is a first-class step, not an afterthought — the Implementation Guide names each role at each stage.

Credentials, drivers, and network — safe by design

Scanning reaches into a live SAP system, so it needs a driver path, a least-privilege SAP login, and network line of sight — never a secret in a script. This solution follows the safe-by-design pattern Microsoft documents:

- **The scan runs on a self-hosted integration runtime (SHIR).** SAP systems are almost always

behind the corporate network, so the Azure (managed) runtime cannot reach them — a **SHIR** installed on a Windows machine with **line of sight to SAP** is required (**verify** the current runtime requirement for your connector).

- **The SAP driver is provided by you, from SAP.** The **SAP JCo** libraries (for ECC/S4HANA/BW) and

the **SAP HANA JDBC driver** (for HANA) are downloaded from the **SAP Support Portal under your SAP license** (an S-user) and placed on the SHIR machine with a supported **Java runtime**. These are **SAP's** deliverables under **your** SAP entitlement — the solution documents where they go and how to verify them; it does not (and cannot) ship SAP's drivers.

- **Authentication uses a least-privilege SAP login held in Azure Key Vault, referenced by name.**

The SAP username/password is stored in an **Azure Key Vault** linked to the Purview account and referenced as a Purview credential **by name** — **no SAP secret is ever placed in a script or document.**

- Scans are always **piloted on a bounded scope first** (one SAP system/client, a scoped set of

tables).

Licensing and the evolving portal

Microsoft Purview Data Governance (the Data Map and its connectors) is billed through a **consumption / pay-as-you-go** model tied to an **Azure subscription**, and the capability set is evolving — the **SAP** connectors, the supported SAP versions, the driver/runtime requirements, and the exact configuration may be in preview or may have changed name, path, or availability. **Verify current pricing, availability, supported versions, driver requirements, and portal paths for your tenant** in Microsoft's documentation and the Purview portal before you plan. This solution cites the method; it never asserts a price, a driver version, or a portal path as permanent.

What this is not

This is the *SAP data-source governance* solution. It is **not** the full Data Map / Unified Catalog build, and it is **not** the Microsoft 365 compliance configuration (sensitivity/retention labels, DLP, insider risk) — those are separate Purview solutions in this category. It complements them by pulling SAP into the same governed map.

The deployment model

> *Assess SAP readiness (connector availability, SHIR, drivers, SAP login, network), design the collection and role model and the driver/credential path, configure the SAP source registration, **pilot scanning on a bounded scope** (one system/client, a scoped table set), validate the results against the gate, then expand across the SAP estate — moving scans from one-time to scheduled only after the pilot passes. The repeatable way: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand.***

2. Requirements Guide — SAP Data Governance: Microsoft Purview

Verify every requirement before you register an SAP source or run a scan. Each entry gives Description, Why, How to verify, and How to implement if missing. Category-wide prerequisites (tenant, roles, toolchain, scope, stakeholders, pilot) are in `global-requirements-purview.md` — this guide adds the prerequisites specific to **governing SAP in the Purview Data Map**.

> ***Licensing/availability caveat.** Microsoft Purview Data Governance is billed by a consumption / pay-as-you-go model on an Azure subscription and is evolving; the **SAP** source connectors (historically **SAP ECC**, **SAP S/4HANA**, **SAP BW**, **SAP HANA**), the supported SAP versions, and the exact driver/runtime/credential configuration may be in preview or may have changed. Where this guide names a capability, a driver, or a portal path, **verify its current availability for your tenant** in Microsoft's documentation and the Purview portal before relying on it.*

2.1 A Microsoft Purview account (Data Map) provisioned on an Azure subscription

Description: A Microsoft Purview account — the Data Map resource — provisioned in an **Azure subscription** (distinct from the M365 compliance portal). The account exposes the collection hierarchy, scanning, and the connectors SAP sources register into. Why: The Data Map and scans live in this

account; without it there is nothing to register SAP sources into, and the SAP connectors are surfaced by this account. How to verify: The account is visible in the Azure portal and its data-plane endpoint (`https://<account>.purview.azure.com`) resolves; run `DGSAP-Discovery.ps1` to confirm reachability. How to implement if missing: Provision a Microsoft Purview account under change control on the target subscription; confirm the consumption billing model and region with your Azure owner.

2.2 SAP connector availability confirmed for your tenant (ECC / S/4HANA / BW / HANA)

Description: Confirmation that the SAP source connector you need — **SAP ECC**, **SAP S/4HANA**, **SAP BW**, or **SAP HANA** — is available in your Purview account, and the current supported **SAP version list** for that connector. Why: Connector availability and supported versions are evolving/preview; you cannot register a source type the connector doesn't currently offer in your tenant. How to verify: In the Purview portal, **Data Map** → **Sources** → **Register**, confirm the SAP source type(s) appear as registerable; check Microsoft's current connector documentation for the supported SAP versions. How to implement if missing: If a connector isn't offered, verify licensing/region and Microsoft's current availability; scope the pilot to the SAP source type actually available and defer the others.

2.3 Least-privilege data-governance roles on the Data Map collection

Description: Role assignments at the right scope — **Collection admin**, **Data source admin**, **Data curator**, **Data reader** — on the collection the SAP sources will register into. Azure **Owner/Contributor** on the account only where account-level changes are required. Why: Purview uses fine-grained, collection-scoped roles; registering a source and running a scan requires **Data source admin** on that collection, and least-privilege is a governance requirement in its own right. How to verify: In the Purview portal, open the collection's **Role assignments**; run `DGSAP-Discovery.ps1` (reports collections and role coverage, read-only). How to implement if missing: Assign the least-privilege role at the narrowest collection scope under change control; reserve account-level Collection admin for a small named group.

2.4 An SAP system inventory in scope, with owners

Description: A written list of the **SAP systems** to govern — each system's **type** (ECC / S/4HANA / BW / HANA), **SID / application-server host / instance number / client** (or HANA host/port/tenant database), and the tables/objects intent — each with a named **data owner** and **data steward**. Why: You register, scan, and classify per source; classification and ownership are meaningless without a known owner per system, and the system details drive the connector and credential configuration. How to verify: For each in-scope SAP system you can name the type, connection details, and its owner; run `DGSAP-Set-Scope.ps1` to record the pilot source set. How to implement if missing: Inventory the SAP landscape with the SAP Basis team, and assign a data owner and steward to every system before registering it.

2.5 A self-hosted integration runtime (SHIR) with line of sight to SAP

Description: A **self-hosted integration runtime** installed on a **Windows machine that has network line of sight to the SAP system** (the application server for ECC/S4HANA/BW, or the HANA host/port), registered to the Purview account. SAP is behind the corporate network, so the Azure (managed) runtime generally cannot reach it — a SHIR is required (**verify** the current runtime requirement for your connector).

Why: A scan cannot classify what it cannot reach; the SHIR is the bridge from Purview to the on-network SAP system, and it is where the SAP driver and Java runtime live. How to verify: The SHIR shows **Running/Available** in the portal; `DGSAP-Discovery.ps1` reports registered integration runtimes; the SHIR machine can reach the SAP host/port (e.g. RFC gateway `33NN`, or HANA SQL port `3NN15`). How to implement if missing: Install and register a SHIR on a hardened Windows host with line of sight to SAP, size it per Microsoft's guidance, and confirm it shows Running/Available before scanning.

2.6 The SAP driver and Java runtime on the SHIR machine (your SAP entitlement)

Description: The **SAP-provided driver** placed on the SHIR machine: for **ECC/S4HANA/BW**, the **SAP JCo** libraries (`sapjco3.jar` + `sapjco3.dll`, 64-bit) plus a supported **64-bit Java runtime (JRE/JDK)** with `JAVA_HOME`/path configured; for **SAP HANA**, the **SAP HANA JDBC driver (**`ngdbc.jar`**)**. These are downloaded from the **SAP Support Portal under your SAP license (an S-user)** — they are SAP's deliverables under your entitlement. Why: Purview's SAP scanner loads these drivers to talk to SAP; without the correct driver + Java version on the SHIR, the scan cannot connect. **Verify** the exact driver and Java versions Microsoft's current connector documentation requires — they change. How to verify: `sapjco3.jar`/`sapjco3.dll` (or `ngdbc.jar`) are present in the location Microsoft's docs specify, the Java runtime version matches, and the SAP source's **Test connection** succeeds; `DGSAP-Test-Readiness.ps1` records a driver-placement checklist item. How to implement if missing: Download the driver from the SAP Support Portal with your S-user, install the required Java runtime on the SHIR machine, place the files per Microsoft's docs, and re-test. The solution does not (and cannot) ship SAP's drivers — obtain them under your SAP license.

2.7 A least-privilege SAP read login, secret in Azure Key Vault — never in code

Description: An **SAP service/read user** with the **least-privilege authorizations** the scan needs — for ECC/S4HANA/BW, authorization to execute the RFC function modules the scanner calls (authorization object `S_RFC`) and read the **data-dictionary metadata** it enumerates; for HANA, a database user with **read** on the schemas in scope. The user's password is stored in an **Azure Key Vault** linked to the Purview account and referenced **by name**. **No credential is ever placed in a script or document.** Why: SAP scans authenticate as this user; storing the secret in Key Vault and referencing it by name is mandatory and is how the provided scripts stay secret-free. Least privilege keeps a governance scanner from carrying more SAP authority than reading metadata requires. How to verify: The SAP user exists with a scoped authorization role, a Key Vault is linked to the Purview account and holds the SAP secret, and the SAP source's **Test connection** succeeds using the named credential. How to implement if missing: With the SAP security team, create a service user and a **least-privilege authorization role** (the `S_RFC` function-module set and metadata-table read the scanner requires — **verify the exact authorizations against Microsoft's current connector documentation**); store the secret in Key Vault; create a Purview credential referencing it by name; verify **Test connection**.

2.8 A classification approach (system + any custom classifications)

Description: A decision on how SAP assets will be classified: rely on the **built-in system classifications** (credit card, national ID, and the large built-in set) and define any **custom classification rules** the organization needs (for example an internal customer- or vendor-ID pattern) via the scan rule set. Why:

Classification is what turns the raw SAP metadata inventory into a sensitivity-aware map that drives ownership, access, and downstream protection. How to verify: System classifications appear on a test scan result; any custom classification rule is defined and matches sample data. How to implement if missing: Start with system classifications for the pilot; define custom rules only where a real internal pattern isn't covered, and validate each against sample data before estate-wide use.

2.9 A pilot scope and change control

Description: A **bounded pilot** — one collection and one SAP system/client with a scoped table/object set — to prove registration, driver/credential/network, scanning, and classification before SAP-wide rollout, plus a change-control window (scans consume SHIR/SAP resources and read a live SAP system). Why: Every solution in this category follows Assess → Design → Configure → **Pilot** → Validate → Expand; scanning is resource- and access-sensitive and is always proven on a bounded scope first. How to verify: A named pilot collection, its SAP source, and a change window exist; run `DGSAP-Set-Scope.ps1` to materialize the pilot scope. How to implement if missing: Choose one well-understood, non-production-critical SAP system (or a low-impact client) as the pilot; agree the scan window and expected SAP load with the SAP Basis team.

Checklist

- Purview account (Data Map) provisioned on an Azure subscription — `DGSAP-Discovery.ps1`
- SAP connector availability + supported SAP versions confirmed for the tenant (ECC/S4HANA/BW/HANA)
- Least-privilege collection roles assigned (Data source admin to register/scan) — `DGSAP-Discovery.ps1`
- SAP systems inventoried with type, connection details, and owners — `DGSAP-Set-Scope.ps1`
- SHIR installed with line of sight to SAP, Running/Available — `DGSAP-Discovery.ps1`
- SAP driver (JCo `sapjco3.*` / HANA `ngdbc.jar`) + Java runtime placed on the SHIR (your SAP license)
- Least-privilege SAP read user created; secret in Azure Key Vault, referenced by name (no secrets in code)
- Classification approach agreed (system + any custom)
- Pilot scope and scan change window agreed — `DGSAP-Set-Scope.ps1`

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell***

*(`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

• **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

• **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

• **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

• **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

• **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

• **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.

• **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.

• **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

• **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

• **Why required:** Configuration, automation, and the provided scripts run through these.

• **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.

• **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

• **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams, endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

• **Why required:** Purview policies key off the in-scope workloads and the classification scheme.

• **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with stakeholders.

• **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

• **Description:** Engagement with the **compliance, records-management, privacy, security, and legal** stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

• **Why required:** Purview configuration must align to enterprise policy and legal/records requirements, not technical defaults; several capabilities have HR/legal sensitivity.

• **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.

• **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

• **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

• **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate → Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

• **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change window/approval path.

• **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — SAP Data Governance: Microsoft Purview

Follow these steps in order. This guide assesses SAP readiness, designs the collection and role model, stands up the SHIR + SAP driver path, establishes the SAP credential safely, registers and **pilot-scans** a bounded SAP source, reviews classifications, then expands across the SAP estate. Artifact names are fixed — create them with the exact names shown and reuse them everywhere. **Every scan is piloted on a bounded scope before the SAP estate is scanned.**

> *Model: prove the SAP connector, driver/credential/network path, and classification on a pilot scope, then expand. The repeatable way: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand.** Verify current portal paths, connector availability, supported SAP versions, and driver/Java requirements for your tenant as you go — the connectors are evolving.*

Step 1 — Assess current state (read-only)

1. Run `DGSAP-Discovery.ps1` to capture readiness signals: the Purview account endpoint is reachable,

which **collections** exist and their **role coverage**, which **integration runtimes** are registered (you need a **SHIR**), and how many SAP (and other) sources are already registered. No changes are made.

1. In the Purview portal (**Data Map** → **Sources** → **Register**) confirm the SAP source type you need

(**SAP ECC / S/4HANA / BW / HANA**) is offered and note the **supported SAP versions** for your tenant.

1. Record gaps against the Requirements checklist. Do **not** proceed past a missing account, a missing

connector, missing least-privilege roles, a missing SHIR, a missing driver, or a missing network path — remediate those first.

Step 2 — Design the collection and role model

1. Decide the **collection** the SAP sources register into. For the pilot, create one collection named

`DGSAP-Collection-Pilot` under the root; plan the broader hierarchy (for example an SAP branch by system or by sensitivity) on paper for the expansion phase.

1. Assign least-privilege roles at the narrowest scope: **Data source admin** on

`DGSAP-Collection-Pilot` to whoever will register/scan SAP sources, **Data curator** to the catalog editors, **Data reader** to browsers, and **Collection admin** to a small named group. Reserve account-level admin for a minimal named group.

Step 3 — Stand up the scanner: SHIR + SAP driver + Java (safe by design)

1. Install and register a **self-hosted integration runtime (SHIR)** on a hardened Windows host with

line of sight to the SAP system (RFC gateway for ECC/S4HANA/BW, or the HANA SQL port). Confirm it shows **Running/Available** in the portal.

1. On the SHIR machine, place the **SAP-provided driver** downloaded from the SAP Support Portal under

your S-user: for ECC/S4HANA/BW, the **SAP JCo** files (`sapjco3.jar` + `sapjco3.dll`, 64-bit) plus a supported **64-bit Java runtime** with `JAVA_HOME`/path set; for HANA, the **HANA JDBC driver** (`ngdbc.jar`). **Verify the exact driver and Java versions Microsoft's current connector docs require** and the folder they must sit in. **Put no secret anywhere** — the driver is not a credential.

1. Confirm the SAP host/port is reachable from the SHIR machine (open only the firewall rules the scanner needs — no broader).

Step 4 — Establish the SAP credential path (Key Vault, safe by design)

1. With the SAP security team, create a **least-privilege SAP read user** and a scoped authorization role (for example SAP user `PURVIEW_SCAN` with role `Z_PURVIEW_SCAN`) carrying the `S_RFC` function-module set and metadata read the scanner requires (for HANA, a DB user with read on the in-scope schemas). **Verify the exact authorizations against Microsoft's current connector documentation.**

1. Link an **Azure Key Vault** to the Purview account (or confirm one is linked), store the SAP user's secret there, and create a Purview credential **by name** (for example `DGSAP-Cred-SAP-KV`) that references the Key Vault secret. This name is what the scripts reference; **the SAP secret is never inlined in a script or document.**

Step 5 — Record the pilot scope

1. Run `DGSAP-Set-Scope.ps1` to record the pilot SAP source (one system/client with a scoped table/object intent) with owner/steward, the pilot collection, the credential name, and the scan-rule-set intent (read-only; produces the scope manifest you register against).

Step 6 — Register the pilot SAP source into the Data Map

1. Register the pilot source into `DGSAP-Collection-Pilot`. You may use the portal (**Register** → **SAP ECC / S/4HANA / BW / HANA**), or run `DGSAP-Register-SapSource.ps1` which **defaults to** `-WhatIf` and prints exactly what it would register (the SAP `kind`, the host/instance/client or HANA host/port, the collection, the SHIR, and the credential name); re-run with `-Commit` to register.

1. Run **Test connection** for the registered source and confirm it passes (the SHIR loads the SAP driver, reaches SAP, and authenticates with the named credential) before scanning. **Do not scan until Test connection passes.**

Step 7 — Configure and pilot the scan (bounded scope, pilot-first)

1. Define the scan configuration: a **scan rule set** named `DGSAP-ScanRuleSet-Pilot` (start from the system default rule set), the **classifications** to apply (system classifications plus any custom rule named `DGSAP-Classification-Custom-<name>`), and a **scoped** scan that targets only the pilot system's intended tables/objects — not the entire SAP dictionary.

1. Trigger the scan **once** on the bounded scope (not scheduled yet) via

`DGSAP-Register-SapSource.ps1 -ConfigureScan` (`-WhatIf` by default; `-Commit` to apply) or the portal. Pass/fail: the scan completes with **status Succeeded** and returns assets with classifications.

1. **Rollback:** if a scan misbehaves, over-reaches, or loads the SAP system, cancel the run in the

portal and set the scan to disabled; a metadata scan does not modify SAP data, but cancel to stop resource consumption. Re-scope and re-run on a smaller set, and coordinate the window with SAP Basis.

Step 8 — Review scanned assets and classifications

1. Run `DGSAP-Report.ps1` to list the pilot SAP assets discovered (tables/objects and, where scanned,

columns/fields), the classifications applied, and any assets returned **unclassified**.

1. With the data owner, confirm the classifications are correct. Where a real internal pattern was

missed, define a **custom classification rule** (`DGSAP-Classification-Custom-<name>`), validate it against sample data, add it to `DGSAP-ScanRuleSet-Pilot`, and re-scan the pilot scope.

Step 9 — Assign ownership on the SAP assets

1. In the catalog, assign the **data owner** and **steward** recorded in the scope manifest to the

scanned SAP source/assets so every governed SAP asset has a named owner — ownership is what makes classification and later access decisions accountable.

Step 10 — Validate against the pilot gate

1. Complete the Testing Guide. Run `DGSAP-Test-Readiness.ps1` for a consolidated PASS/GAP gate check.

The pilot passes only when: the SAP source is registered and **Test connection** passes; the scan **Succeeded** and returned classified assets; and every scanned pilot asset has an owner assigned.

1. Remediate any gaps and re-run the relevant validation before proceeding. Do **not** widen scope

until the gate passes.

Step 11 — Expand across the SAP estate (enforce)

1. After the pilot passes, register further SAP systems one at a time, re-running

`DGSAP-Set-Scope.ps1`, `DGSAP-Register-SapSource.ps1`, and `DGSAP-Report.ps1` for each new source. Reuse the same SHIR/driver pattern and Key Vault credential pattern (a per-system read user where your SAP security model requires it).

1. Move scans from one-time to **scheduled** only once a source is validated, so classifications stay

current as SAP data changes — and coordinate scan windows with SAP Basis to avoid production load.

1. Keep ownership assigned on every newly scanned asset and track coverage with `DGSAP-Report.ps1`.

Order of operations

```
Assess -> Design (collection/roles) -> SHIR + SAP driver + Java -> SAP
credential (Key Vault) & network -> Record scope -> Register SAP source ->
Pilot scan (bounded) -> Review classifications -> Assign ownership -> Validate
gate -> Expand (scheduled scans)
```

5. Testing Guide — SAP Data Governance: Microsoft Purview

Run these tests after the pilot scan (Implementation Steps 6–9) and again after each expansion source. Each test confirms one part of the SAP-governance workflow works end to end. All read-only checks are safe to repeat.

Test 1 — Account and roles are ready

Do: Run `DGSAP-Discovery.ps1`. Expected: The Purview account endpoint is reachable, `DGSAP-Collection-Pilot` exists, and least-privilege roles (Collection admin, Data source admin, Data curator, Data reader) are reported as assigned; **Data source admin** (needed to register/scan) is present. No required role is missing.

Test 2 — The SAP connector is available

Do: In the Purview portal, open **Data Map** → **Sources** → **Register**. Expected: The SAP source type you need (**SAP ECC / S/4HANA / BW / HANA**) appears as a registerable source type for your tenant, and the supported SAP version list matches what the pilot system runs.

Test 3 — The SHIR and SAP driver are in place

Do: Run `DGSAP-Discovery.ps1` and check the SHIR machine. Expected: A **self-hosted integration runtime** is registered and shows **Running/Available**; the SAP-provided driver (`sapjco3.jar` + `sapjco3.dll` for ECC/S4HANA/BW, or `ngdbc.jar` for HANA) and a supported **64-bit Java runtime** are present on the SHIR machine in the location Microsoft's docs specify; the SHIR can reach the SAP host/port.

Test 4 — The SAP credential path works (Key Vault + Test connection)

Do: On the registered SAP source, run **Test connection**; confirm the credential references the Key Vault secret **by name** and the SHIR shows Running/Available. Expected: Purview (via the SHIR) loads the SAP driver, reaches the SAP system, and authenticates using the **named Key Vault credential** with the **least-privilege SAP read user** — **Test connection passes**. No SAP secret is stored anywhere.

Test 5 — The scan succeeds and returns assets

Do: Review the pilot scan run in the portal and run `DGSAP-Report.ps1`. Expected: The scan status is **Succeeded**, the run scanned the intended bounded scope (the pilot system's intended tables/objects, not the entire SAP dictionary), and assets (SAP tables/objects and, where scanned, fields/columns) were returned into the Data Map.

Test 6 — SAP assets are classified

Do: Run `DGSAP-Report.ps1` and review the classification coverage for the pilot SAP assets. Expected: System (and any custom) classifications are applied to the assets that contain matching data; the count of unexpectedly **unclassified** assets is understood and accepted by the data owner.

Test 7 — Scanned SAP assets have owners

Do: Open the pilot SAP source/assets in the catalog and run `DGSAP-Report.ps1`. Expected: Every scanned pilot asset has a named **owner/steward** assigned (from the scope manifest); no governed SAP asset is unowned.

Test 8 — Consolidated readiness gate

Do: Run `DGSAP-Test-Readiness.ps1 -PurviewAccount <account> -Collection DGSAP-Collection-Pilot`. Expected: A single PASS/GAP summary confirming account reachable, roles present, a SHIR registered, the SAP source registered and Test-connection-ready, at least one scan Succeeded, classification coverage

> *0, and owners assigned — the pilot gate in one view.*

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1–7 pass: connector available, SHIR + driver in place, SAP Key Vault credential Test connection passes, scan Succeeded with classified assets, and every scanned asset has an owner (Test 8 shows all PASS)
Expand pass	Tests 1–7 pass for each new SAP source, and scheduled scans are enabled only after the source validates (with scan windows coordinated with SAP Basis)

6. Troubleshooting Guide — SAP Data Governance: Microsoft Purview

Field issues and remediation, in the order you hit them. Read-only checks (`DGSAP-Discovery.ps1`, `DGSAP-Report.ps1`, `DGSAP-Test-Readiness.ps1`) are safe to repeat while you diagnose.

The SAP connector isn't offered in Register

Symptom: **Data Map** → **Sources** → **Register** doesn't list the SAP source type you need (ECC / S/4HANA / BW / HANA). Likely cause: Connector availability is licensing-/region-/preview-dependent and evolving. Fix: Verify your Purview account's region and licensing, and Microsoft's current connector availability for your tenant. Scope the pilot to the SAP source type actually offered and defer the others; do not assume a connector that isn't shown.

`DGSAP-Discovery.ps1` reports the endpoint as GAP (not reachable)

Symptom: The script can't read collections. Likely causes: wrong account name, not signed in, missing Data reader role, or an account API version that has moved. Fix: Confirm the short account name, run `Connect-AzAccount`, ensure at least **Data reader** on the collection, and verify the current account API version for your tenant. The script never stores a secret — it acquires a data-plane token for `https://purview.azure.net`.

No self-hosted integration runtime (SHIR) shows up

Symptom: `DGSAP-Discovery.ps1` / `DGSAP-Test-Readiness.ps1` report no self-hosted IR. Likely cause: SAP is behind the corporate network and the Azure (managed) runtime cannot reach it; a SHIR hasn't been installed/registered. Fix: Install and register a **SHIR** on a hardened Windows host with line of sight to the SAP host/port, confirm it shows **Running/Available**, and re-run discovery. **Verify** the current runtime requirement for your connector.

Test connection fails: SAP driver / Java not found

Symptom: **Test connection** fails referencing the SAP connector, a missing library, or Java. Likely cause: the **SAP JCo** files (`sapjco3.jar` + `sapjco3.dll`, ECC/S4HANA/BW) or the **HANA JDBC driver** (`ngdbc.jar`, HANA), or a supported **64-bit Java runtime**, aren't present on the SHIR machine — or the versions don't match Microsoft's requirement. Fix: Download the driver from the **SAP Support Portal under your S-user**, install the required Java runtime, place the files in the location Microsoft's current connector docs specify, set `JAVA_HOME`/path, restart the SHIR service if required, and re-test. Match the **exact** driver and Java versions Microsoft's docs call for — a version mismatch is the most common failure. The solution cannot ship SAP's drivers; obtain them under your SAP license.

Test connection fails: authentication or authorization (SAP side)

Symptom: The SHIR reaches SAP but authentication or metadata read fails. Likely causes: the SAP read user's password is wrong or expired, the Key Vault credential references the wrong secret, or the user's authorization role is missing the `S_RFC` function-module set or the metadata-table read the scanner needs (for HANA, missing read on the in-scope schemas). Fix: Confirm the Key Vault secret is current and the Purview credential references it **by name**; with the SAP security team, confirm the authorization role grants exactly the least-privilege authorizations Microsoft's current connector docs require — no more. Never place the SAP password in a script or the scan definition.

Test connection fails: network path to SAP

Symptom: The SHIR cannot reach the SAP host/port. Likely cause: firewall rules block the RFC gateway (ECC/S4HANA/BW) or the HANA SQL port from the SHIR machine. Fix: Open only the specific firewall rule the scanner needs from the SHIR to the SAP host/port — no broader. Confirm the SHIR machine can resolve and reach the host, then re-test.

Scan runs but returns no assets (or far fewer than expected)

Symptom: The scan status is Succeeded but `DGSAP-Report.ps1` shows few/no SAP assets. Likely causes: the scan scope was too narrow (or pointed at the wrong tables/objects), or the SAP read user

lacks read on some metadata. Fix: Review the scan's scope and rule set, widen it deliberately to the intended tables/objects (still bounded), confirm the read user can see them, and re-scan the pilot scope.

Scan Succeeded but assets are unclassified

Symptom: Assets appear in the catalog but with no classifications. Likely causes: the data genuinely doesn't match the system classifications, or a real internal pattern (an SAP customer/vendor ID scheme) isn't covered. Fix: Confirm with the data owner whether the assets should carry a classification. Where a real pattern is missed, define a **custom classification rule** (`DGSAP-Classification-Custom-<name>`), validate it against sample data, add it to `DGSAP-ScanRuleSet-Pilot`, and re-scan. Don't over-fit custom rules — trust system classifications first.

The scan loads the SAP system or collides with a batch window

Symptom: SAP Basis reports load during a scan, or the scan overlaps a period-close/batch job. Fix: Keep the scan **one-time and scoped** until validated; schedule scans only after a source passes, and coordinate every window with SAP Basis so a governance scan never competes with production processing. If a run misbehaves, cancel it in the portal and set the scan to disabled (a metadata scan does not modify SAP data) — then re-scope.

`-WhatIf` printed a plan but nothing was registered

Symptom: `DGSAP-Register-SapSource.ps1` printed a plan and made no change. Cause: **This is by design** — the script defaults to `-WhatIf` and only prints the payload it would send. Re-run with `-Commit` once you've confirmed the SAP source `kind`, the property schema, the API version, the SHIR name, and the Key Vault credential name for your tenant.

Roles look wrong or a curator can't classify

Symptom: A user can't register a source, run a scan, or edit classifications. Fix: Confirm least-privilege roles at the collection scope — **Data source admin** to register/scan, **Data curator** to edit metadata/classifications, **Data reader** to browse. Assign the narrowest role that lets the person do their job; reserve Collection/account admin for a small named group.

When to escalate vs. defer

- **Escalate (technical):** connector not offered, SHIR/driver/network failures, scan errors, API-version drift — these are configuration issues this guide's steps and scripts address.
- **Defer (determination):** whether a given SAP source is in scope, how its data must be classified, retained, or accessed, and any regulatory obligation — these belong to the data owners, stewards, and the compliance/records/privacy/legal functions, not to the scanning configuration.

7. Best Practices — SAP Data Governance: Microsoft Purview

- Confirm connector availability before you design anything. The SAP ECC / S/4HANA / BW / HANA

connectors, the supported SAP versions, and the driver/Java requirements are evolving — read them from the Purview registration UI and Microsoft's current docs for your tenant rather than assuming a value that may have moved.

- Scan through a SHIR, not the managed runtime. SAP systems sit behind the corporate network; install

a self-hosted integration runtime on a hardened Windows host with line of sight to SAP, size it per Microsoft's guidance, and confirm Running/Available before you register anything.

- Get the SAP driver from SAP, under your license. The SAP JCo (`sapjco3.jar` + `sapjco3.dll`) and the

HANA JDBC driver (`ngdbc.jar`) come from the SAP Support Portal via your S-user, with a supported 64-bit Java runtime on the SHIR. Match the exact driver and Java versions Microsoft's current connector docs require — a mismatch is the most common Test-connection failure.

- Use a least-privilege SAP read user, and keep its secret in Key Vault by name. Create a dedicated SAP

service user with a scoped authorization role (the `S_RFC` function-module set and metadata read the scanner needs — verify against Microsoft's docs), store its secret in an Azure Key Vault linked to the account, and reference it by name. Never place an SAP password in a script, a scan, or a document; never reuse an SAP admin account for a governance scan.

- Remember it maps metadata, not bulk data. SAP scans catalog and classify the metadata (systems,

tables, fields) and sample to classify — set that expectation with SAP owners so a governance scan is never mistaken for a data extraction.

- Pilot every scan on a bounded scope. A scoped, one-time scan on one non-production SAP system/client

proves classification, connectivity, and SAP load before you point a scan at the estate; move to scheduled scans only after a source validates, and coordinate every window with SAP Basis.

- Mind SAP system load and timing. Scanning reads the SAP dictionary and executes RFC calls; scope the

scan, keep it one-time until validated, and agree the window with the SAP Basis team so a scan never competes with a batch or period-close.

- Trust system classifications first, add custom sparingly. The built-in classifications cover most

regulated data; define a custom classification rule (`DGSAP-Classification-Custom-<name>`) only for a real internal pattern (an SAP customer- or vendor-ID scheme, for example), and validate it against sample data before SAP-wide use.

- Every scanned SAP asset gets an owner and a steward. Ownership is what makes classification and access

decisions accountable; an unowned SAP source in the catalog is a governance gap, not a governed asset.

- Treat determinations as someone else's job. The solution registers, scans, classifies, and catalogs

SAP; the business/records/legal owners decide sensitivity, retention, and access. Keep that separation explicit so technical scanning is never mistaken for a compliance ruling.

• Verify licensing, pricing, and portal paths as you go. The connectors and the consumption billing are evolving — confirm current availability and pricing in Microsoft's docs and the portal rather than assuming a path, a driver version, or a price is permanent.

8. Deliverables Summary — SAP Data Governance: Microsoft Purview

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	Registering, scanning, and classifying SAP into the Purview Data Map — the SAP connectors, the safe SHIR/driver/credential model, and how it complements M365 Purview (the why)
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Purview)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps: design, SHIR + SAP driver, SAP credential, register, pilot-scan, classify, assign ownership, validate, expand
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

Named artifacts

Artifact	Name
Pilot collection	`DGSAP-Collection-Pilot`
SAP credential (references the Key Vault secret by name)	`DGSAP-Cred-SAP-KV`
Scan rule set	`DGSAP-ScanRuleSet-Pilot`
Custom classification rule (as needed)	`DGSAP-Classification-Custom-<name>`
Example SAP read user (created in SAP)	`PURVIEW_SCAN`
Example SAP authorization role (created in SAP)	`Z_PURVIEW_SCAN`

PowerShell scripts

Script	What it does
`DGSAP-Discovery.ps1`	Reports readiness: account endpoint reachable, collections + role coverage, registered integration runtimes (looks for a SHIR), and registered SAP source counts (Purview data-plane REST via an Azure token; read-only)
`DGSAP-Set-Scope.ps1`	Records the pilot/expansion SAP scope — the SAP system(s), type, connection details, owners, collection, credential name, and scan-rule-set intent — as a scope manifest to configure against (read-only)
`DGSAP-Register-SapSource.ps1`	Registers an SAP ECC/S4HANA/BW/HANA source (via the SHIR) and configures a scoped scan; **defaults to** `-WhatIf` (prints the plan), applies only with `-Commit`; credential referenced by name — no secrets in code
`DGSAP-Report.ps1`	Scan + classification coverage report for SAP assets: scan run status per source, classified vs unclassified assets, and owner coverage (read-only)
`DGSAP-Test-Readiness.ps1`	Consolidated PASS/GAP pilot-gate check across account, roles, SHIR, SAP source registration/Test-connection readiness, scan status, classification coverage, and ownership (read-only)

What the buyer receives

- This implementation guide as a Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable method following the model: Assess → Design → Configure → Pilot → Validate → Expand.

Outcome

SAP data governed from Purview — SAP systems registered into the Data Map, scanned and classified, and owned — proven on a bounded pilot before you scale, with the SAP blind spot closed and regulatory, records, and access determinations left to the owners who make them.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

DGSAP-Discovery.ps1

Purpose: Reports readiness so you can assess current state before registering SAP sources or scanning: whether the Microsoft Purview account data-plane endpoint is reachable, which collections exist and their role-assignment coverage (especially Data source admin, needed to register/scan), which integration runtimes are registered (SAP scanning needs a self-hosted integration runtime, SHIR), and how many SAP sources are already registered. Read-only — it makes no change to the account, sources, or catalog. Supports the Assess step. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DGSAP-Discovery.ps1 -PurviewAccount contoso-purview Requires the Az PowerShell modules (Az.Accounts). Sign in first with Connect-AzAccount using an account holding at least Data reader on the collections. The script acquires a data-plane token for the Purview REST API (resource https://purview.azure.net); no secret is stored in the script. EXPECTED RESULTS A readiness summary: endpoint reachable (PASS/GAP), collection count and per-collection role coverage, registered integration runtimes (with a self-hosted-IR PASS/GAP for SAP), and a count of registered SAP sources — with a PASS/GAP line per signal to record against the Requirements checklist. .NOTES Purview — SAP Data Governance. Read-only against the Purview data-plane REST API. Verify your licensing/tenant: the Data Governance experience, its endpoints, and the SAP connectors are evolving. > [CmdletBinding()] param([Parameter(Mandatory = $true)] [string]$PurviewAccount, # short account name, e.g. contoso-purview [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

How to run: powershell -ExecutionPolicy Bypass -File .\DGSAP-Discovery.ps1 -PurviewAccount contoso-purview Requires the Az PowerShell modules (Az.Accounts). Sign in first with Connect-AzAccount using an account holding at least Data reader on the collections. The script acquires a data-plane token for the Purview REST API (resource https://purview.azure.net); no secret is stored in the script. EXPECTED RESULTS A readiness summary: endpoint reachable (PASS/GAP), collection count and per-collection role coverage, registered integration runtimes (with a self-hosted-IR PASS/GAP for SAP), and a count of registered SAP sources — with a PASS/GAP line per signal to record against the Requirements checklist. .NOTES Purview — SAP Data Governance. Read-only against the Purview data-plane REST API. Verify your licensing/tenant: the Data Governance experience, its endpoints, and the SAP connectors are evolving. > [CmdletBinding()] param([Parameter(Mandatory = $true)] [string]$PurviewAccount, # short account name, e.g. contoso-purview [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

DGSAP-Register-SapSource.ps1

Purpose: Registers an SAP source (ECC / S/4HANA / BW / HANA) into a Data Map collection and (with -ConfigureScan) prepares a scoped, one-time scan through a self-hosted integration runtime (SHIR) using a named scan rule set. SAFE BY DEFAULT: with no -Commit it runs in -WhatIf mode and only prints the exact registration/scan payload it WOULD send — it changes nothing. Pass -Commit to apply. No credential is ever placed in this script; SAP authentication uses an Azure Key Vault credential referenced by name. HOW TO RUN Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\DGSAP-Register-SapSource.ps1 -PurviewAccount contoso-purview `-Collection DGSAP-Collection-Pilot -SourceName sap-ecc-01 -Kind SapEcc` -SapHost erp-app-01.contoso.example -SystemNumber 00 -Client 100 -Shir DGSAP-SHIR-01 Apply: ... same args

... -ConfigureScan -ScanRuleSet DGSAP-ScanRuleSet-Pilot ` -KeyVaultCredential DGSAP-Cred-SAP-KV -Commit Requires Az.Accounts and Connect-AzAccount with Data source admin on the collection. EXPECTED RESULTS Default: a printed "[WhatIf] would register / would configure scan" plan and NO change. With -Commit: the source is registered (HTTP 200/201) and, with -ConfigureScan, a scan definition is created through the named SHIR; the script prints the resulting object names. Trigger/monitor the scan run in the portal or with DGSAP-Report.ps1. .NOTES Purview — SAP Data Governance. Writes gated behind -Commit; -WhatIf is the default. Verify the exact source "kind" (e.g. SapEcc / SapS4Hana / SapBw / SapHana), the scan API version, the SHIR requirement, and the SAP driver/Java prerequisites for your connector in Microsoft's current docs. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [Parameter(Mandatory = $true)][string]$Collection, [Parameter(Mandatory = $true)][string]$SourceName, [Parameter(Mandatory = $true)][string]$Kind, # SapEcc | SapS4Hana | SapBw | SapHana — verify per connector [string]$SapHost, # application-server host (ECC/S4/BW) or HANA host [string]$SystemNumber, # SAP instance/system number, e.g. 00 (ECC/S4/BW) [string]$Client, # SAP client, e.g. 100 (ECC/S4/BW) [string]$HanaPort, # HANA SQL port, e.g. 30015 (HANA) [string]$Shir, # NAME of the self-hosted integration runtime to scan through [switch]$ConfigureScan, [string]$ScanRuleSet = 'DGSAP-ScanRuleSet-Pilot', [string]$KeyVaultCredential, # NAME of a Key Vault credential already linked to the account [string]$Endpoint = "https://$PurviewAccount.purview.azure.com", [switch]$Commit)

How to run: Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\DGSAP-Register-SapSource.ps1 -PurviewAccount contoso-purview `-Collection DGSAP-Collection-Pilot -SourceName sap-ecc-01 -Kind SapEcc` -SapHost erp-app-01.contoso.example -SystemNumber 00 -Client 100 -Shir DGSAP-SHIR-01 Apply: ... same args ... -ConfigureScan -ScanRuleSet DGSAP-ScanRuleSet-Pilot ` -KeyVaultCredential DGSAP-Cred-SAP-KV -Commit Requires Az.Accounts and Connect-AzAccount with Data source admin on the collection. EXPECTED RESULTS Default: a printed "[WhatIf] would register / would configure scan" plan and NO change. With -Commit: the source is registered (HTTP 200/201) and, with -ConfigureScan, a scan definition is created through the named SHIR; the script prints the resulting object names. Trigger/monitor the scan run in the portal or with DGSAP-Report.ps1. .NOTES Purview — SAP Data Governance. Writes gated behind -Commit; -WhatIf is the default. Verify the exact source "kind" (e.g. SapEcc / SapS4Hana / SapBw / SapHana), the scan API version, the SHIR requirement, and the SAP driver/Java prerequisites for your connector in Microsoft's current docs. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [Parameter(Mandatory = $true)][string]$Collection, [Parameter(Mandatory = $true)][string]$SourceName, [Parameter(Mandatory = $true)][string]$Kind, # SapEcc | SapS4Hana | SapBw | SapHana — verify per connector [string]$SapHost, # application-server host (ECC/S4/BW) or HANA host [string]$SystemNumber, # SAP instance/system number, e.g. 00 (ECC/S4/BW) [string]$Client, # SAP client, e.g. 100 (ECC/S4/BW) [string]$HanaPort, # HANA SQL port, e.g. 30015 (HANA) [string]$Shir, # NAME of the self-hosted integration runtime to scan through [switch]$ConfigureScan, [string]$ScanRuleSet = 'DGSAP-ScanRuleSet-Pilot', [string]$KeyVaultCredential, # NAME of a Key Vault credential already linked to the account [string]$Endpoint = "https://$PurviewAccount.purview.azure.com", [switch]$Commit)

DGSAP-Report.ps1

Purpose: Reports the health of the governed SAP estate for a scope: latest scan run status per SAP source, counts of classified vs unclassified SAP assets, and ownership coverage (assets with owners). Read-only — it queries the Purview data-plane REST / catalog search and changes nothing. Use it to review the pilot (Testing Guide) and to track coverage as the SAP estate expands. HOW TO RUN

powershell -ExecutionPolicy Bypass -File .\DGSAP-Report.ps1 -PurviewAccount contoso-purview ` -Collection DGSAP-Collection-Pilot Requires Az.Accounts + Connect-AzAccount with at least Data reader on the collection. EXPECTED RESULTS A coverage summary: per-SAP-source last scan status (Succeeded/Failed/Running), classified vs unclassified asset counts with a classification-coverage %, and an ownership line (assets with an owner assigned). PASS/GAP flags you can record against the pass/fail gate. .NOTES Purview — SAP Data Governance. Read-only. Verify current search/scan API versions for your tenant; the Data Governance experience and its endpoints are evolving. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [string]$Collection, [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

How to run: powershell -ExecutionPolicy Bypass -File .\DGSAP-Report.ps1 -PurviewAccount contoso-purview ` -Collection DGSAP-Collection-Pilot Requires Az.Accounts + Connect-AzAccount with at least Data reader on the collection. EXPECTED RESULTS A coverage summary: per-SAP-source last scan status (Succeeded/Failed/Running), classified vs unclassified asset counts with a classification-coverage %, and an ownership line (assets with an owner assigned). PASS/GAP flags you can record against the pass/fail gate. .NOTES Purview — SAP Data Governance. Read-only. Verify current search/scan API versions for your tenant; the Data Governance experience and its endpoints are evolving. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [string]$Collection, [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

DGSAP-Set-Scope.ps1

Purpose: Captures and writes a scope manifest for the SAP pilot (or an expansion wave): the target collection, the SAP system(s) to register (type, connection details, owner/steward, the SHIR to scan through, and the credential name each uses), and the scan-rule-set intent. Read-only against the tenant — it writes only a local JSON manifest you register and scan against; it does not change the Purview account or any SAP system, and it holds NO SAP secret (credentials are referenced by name). HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DGSAP-Set-Scope.ps1 -Collection DGSAP-Collection-Pilot `-SapSystem @('ECC:erp-app-01.contoso.example:00:100','HANA:hana-01.contoso.example:30015:HDB')` -Shir DGSAP-SHIR-01 -Credential DGSAP-Cred-SAP-KV ` -OutFile .\sap-pilot-scope.json Each -SapSystem entry is 'TYPE:host:instanceOrPort:clientOrTenant' (TYPE = ECC | S4HANA | BW | HANA — verify the exact source kind for your connector). EXPECTED RESULTS A sap-pilot-scope.json manifest listing the collection, the SAP systems (each with type, connection details, owner/steward placeholders, the SHIR, and the credential name), and the scan rule set, plus a printed summary — used as the input to DGSAP-Register-SapSource.ps1. .NOTES Purview — SAP Data Governance. Local manifest only; no tenant or SAP change; no secrets. Names are white-label placeholders (contoso.example). Assign a real owner/steward before registering. > [CmdletBinding()] param([string]$Collection = 'DGSAP-Collection-Pilot', [string[]]$SapSystem = @(), # 'TYPE:host:instanceOrPort:clientOrTenant' [string]$Shir = 'DGSAP-SHIR-01', # NAME of the self-hosted integration runtime to scan through [string]$Credential = 'DGSAP-Cred-SAP-KV', # NAME of the Purview credential referencing the Key Vault secret [string]$ScanRuleSet = 'DGSAP-ScanRuleSet-Pilot', [string]$OutFile = '.\sap-pilot-scope.json')

How to run: powershell -ExecutionPolicy Bypass -File .\DGSAP-Set-Scope.ps1 -Collection DGSAP-Collection-Pilot `-SapSystem @('ECC:erp-app-01.contoso.example:00:100','HANA:hana-01.contoso.example:30015:HDB')` -Shir DGSAP-SHIR-01 -Credential DGSAP-Cred-SAP-KV ` -OutFile .\sap-pilot-scope.json Each -SapSystem entry is 'TYPE:host:instanceOrPort:clientOrTenant' (TYPE = ECC | S4HANA | BW | HANA — verify the exact source kind for your connector). EXPECTED RESULTS A sap-pilot-scope.json manifest listing the collection, the SAP systems (each with type,

connection details, owner/steward placeholders, the SHIR, and the credential name), and the scan rule set, plus a printed summary — used as the input to DGSAP-Register-SapSource.ps1. .NOTES Purview — SAP Data Governance. Local manifest only; no tenant or SAP change; no secrets. Names are white-label placeholders (contoso.example). Assign a real owner/steward before registering. > [CmdletBinding()] param([string]$Collection = 'DGSAP-Collection-Pilot', [string[]]$SapSystem = @(), # 'TYPE:host:instanceOrPort:clientOrTenant' [string]$Shir = 'DGSAP-SHIR-01', # NAME of the self-hosted integration runtime to scan through [string]$Credential = 'DGSAP-Cred-SAP-KV', # NAME of the Purview credential referencing the Key Vault secret [string]$ScanRuleSet = 'DGSAP-ScanRuleSet-Pilot', [string]$OutFile = '.\sap-pilot-scope.json')

DGSAP-Test-Readiness.ps1

Purpose: Runs the pilot gate in one view: confirms the Purview account endpoint is reachable, the named pilot collection exists with least-privilege roles, a self-hosted integration runtime (SHIR) is registered (SAP scans need one), the SAP source is registered, at least one scan has Succeeded, classification coverage is > 0, and scanned SAP assets have owners. Read-only — it changes nothing. Use it as the Step 10 gate before widening scope. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DGSAP-Test-Readiness.ps1 -PurviewAccount contoso-purview ` -Collection DGSAP-Collection-Pilot Requires Az.Accounts + Connect-AzAccount with at least Data reader on the collection. EXPECTED RESULTS A single PASS/GAP summary across account reachability, roles, SHIR, SAP source registration, scan status, classification coverage, and ownership — plus an overall pilot-gate PASS/GAP line. Any GAP must be remediated before expanding beyond the pilot. .NOTES Purview — SAP Data Governance. Read-only. Test connection on the SAP source is confirmed in the portal (driver/network/credential) — this script reports registration + scan + coverage signals. Verify current Purview data-plane API versions for your tenant. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [string]$Collection = 'DGSAP-Collection-Pilot', [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

How to run: powershell -ExecutionPolicy Bypass -File .\DGSAP-Test-Readiness.ps1 -PurviewAccount contoso-purview ` -Collection DGSAP-Collection-Pilot Requires Az.Accounts + Connect-AzAccount with at least Data reader on the collection. EXPECTED RESULTS A single PASS/GAP summary across account reachability, roles, SHIR, SAP source registration, scan status, classification coverage, and ownership — plus an overall pilot-gate PASS/GAP line. Any GAP must be remediated before expanding beyond the pilot. .NOTES Purview — SAP Data Governance. Read-only. Test connection on the SAP source is confirmed in the portal (driver/network/credential) — this script reports registration + scan + coverage signals. Verify current Purview data-plane API versions for your tenant. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [string]$Collection = 'DGSAP-Collection-Pilot', [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")